

Sistemas Multiagente

Soluciones

1. Uno

El hecho de tener acciones que permiten cambiar el orden en que los jugadores juegan no cambia la formulación matemática del valor de una estrategia conjunta π , que notamos V_π , para juegos alternados. En la definición de juego alternado se asume la existencia de una función “Player” que determina que jugador juega en cada estado. Esta es parte del modelo del juego.

Para el caso del ejercicio, supongamos que el conjunto de jugadores P es $\{0, 1, \dots, N-1\}$, siendo N la cantidad de jugadores, y están ubicados de tal manera que $(i+1) \bmod N$ está a la derecha de i , asumiendo que el sentido de la ronda inicial es hacia la derecha. Esto es para visualizar el juego. Las acciones son las cartas.

En el estado es necesario tener información del sentido de la vuelta. Para esto tenemos en el estado una variable, **sense**, que indica el sentido de giro: -1 (hacia la izquierda) o 1 (hacia la derecha). Teniendo en cuenta lo dicho anteriormente, el valor de **sense** en el estado inicial s_{init} es 1, es decir, $s_{init}(\text{sense}) = 1$.

Además, tenemos en el estado el último jugador que jugó, denominado **last-player**, y la última carta jugada, es decir, la que se encuentra arriba de la pila de descarte, denominada **top-card**. En el estado inicial s_{init} ambas están indefinidas.

Estas variables se actualizan al calcular el estado siguiente $s' = \text{Suc}(s, a)$:

- $s'(\text{top-card}) = a$,
- $s'(\text{last-player}) = \text{Player}(s)$, que se define más abajo, y
- $s'(\text{sense}) = -1 \text{ } s(\text{sense})$ si $a = \text{“dar vuelta la ronda”}$, si no $s'(\text{sense}) = s(\text{sense})$.

Asumimos que el jugador 0 es el que comienza, entonces $\text{Player}(s_{init}) = 0$. Para el resto de los estados s , definimos $\text{Player}(s)$ de la siguiente manera (obviamos los casos de borde):

- Si $s(\text{top-card}) = \text{“saltar al siguiente jugador”}$:
$$\text{Player}(s) = (s(\text{last-player}) + 2 \text{ } s(\text{sense})) \bmod N.$$
- Si $s(\text{top-card}) = \text{“jugar de nuevo”}$:
$$\text{Player}(s) = s(\text{last-player})$$
- En cualquier otro caso de $s(\text{top-card})$:
$$\text{Player}(s) = (s(\text{last-player}) + 1 \text{ } s(\text{sense})) \bmod N$$

No es necesario diferenciar si **top-card** es “dar vuelta la ronda” porque el efecto de esta carta en el sentido se tomó en cuenta en **Suc**.

Otra formulación posible es usar **current-player**, en lugar de **last-player**. En este caso, $\text{Player}(s) = \text{current-player}(s)$ y hay que definir $s' = \text{Suc}(s, a)$ apropiadamente.

2. Blokus

Definamos la función $C_p(s)$ que determina la cantidad de cuadrados que tiene el jugador p en el estado s . Usando C , definimos Eval de la siguiente manera:

$$\text{Eval}(s) = C_o(s) - C_a(s)$$

dónde “o” representa al oponente y “a” al agente.

Definimos:

$$\text{Puntos}_p(s) = -C_p(s) + \text{bono}_p(s)$$

Entonces:

$$\text{Puntos}_a(s) - \text{Puntos}_o(s) = (C_o(s) - C_a(s)) + (\text{bono}_a(s) - \text{bono}_o(s))$$

Es decir:

$$\text{Puntos}_a(s) - \text{Puntos}_o(s) = \text{Eval}(s) + (\text{bono}_a(s) - \text{bono}_o(s))$$

Sea s un estado final. Decimos que s es “win” para el agente, si

$$\text{Puntos}_a(s) > \text{Puntos}_o(s).$$

En este caso:

$$\text{Utilidad}_a(\text{win}) = 1.$$

De la misma manera, decimos “draw” si son iguales y “loss” si es menor, y definimos Utilidad apropiadamente para un juego de suma 0.

Ahora veamos que Eval cumple la primera propiedad, es decir, ordena los estados igual que Utilidad . Si s es “win”, entonces $\text{Eval}(s) \geq 0$ (podría ser 0 si ambos jugadores colocaron todas sus fichas). Si s es “draw”, entonces $\text{Eval}(s) = 0$. Si s es “loss”, tenemos que $\text{Eval}(s) \leq 0$. Por lo tanto, $\text{Eval}(\text{win}) \geq \text{Eval}(\text{draw}) \geq \text{Eval}(\text{loss})$.

Nótese que si el juego terminara cuando solo un jugador coloca todas sus fichas (este jugador gana) o cuando ninguno puede colocar ninguna (nadie recibe bono), entonces Eval cumple con el orden total (porque solo puede ser 0 en “draw”). También se puede ver que Eval desalienta jugar el monómino porque solo contribuye a lo sumo en 1.

Claramente, Eval se puede calcular de manera rápida, incluso de manera incremental, haciendo la actualización apropiada en cada jugada, es decir, iniciando con un valor igual a la suma de todos los cuadrados y descontando la cantidad de cuadrados de la pieza jugada en cada turno.

Además, se cumple que Eval está razonablemente correlacionada con las posibilidades reales de ganar dado que, si en un episodio, el agente mantiene Eval siempre estrictamente positiva entonces gana.

3. Pacman

Para que el juego pueda ser resuelto con Minimax hay que transformarlo a un juego de dos jugadores, **P** y **G**.

El jugador **P** es Pacman. El jugador **G**, se forma componiendo todos los fantasmas inteligentes, es decir, $\mathbf{G} = (g_1, \dots, g_N)$. Los fantasmas random son parte del estado, pero no son jugadores. Esto es igual que en el caso del obligatorio.

Las acciones de **P** son las de Pacman. Las acciones de **G** son vectores de acciones de los fantasmas inteligentes. Es decir, se piensa el juego como si todos los fantasmas jugaran simultáneamente.

La regla de fin se redefine de manera natural considerando que **G** come a **P** cuando algún fantasma inteligente g_i come a Pacman.

La función de Utilidad es tal que $Utilidad_P$ es igual a la utilidad de Pacman y $Utilidad_G$ es la suma de las utilidades de los fantasmas inteligentes, es decir $\sum 1/N = 1$, si comen a Pacman y $-\sum 1/N = -1$, si pierden, y 0 si hay empate, manteniéndose la propiedad de juego de suma 0.